

Chapter 1 计算机抽象

1. 计算机性能

(1) 计算机性能的评测指标

① time to do tasks 执行时间

② Tasks per day 吞吐/带宽

→ 两者无关系, 有时相关, 有时矛盾, 视情况而定

(2) CPU性能分析的基本公式

① CPU时间: CPU执行给定程序的时间, 不包含 I/O or INT

② CPU Time = Instruction count $\times$ CPI $\times$ Cycle time
→ 每条指令的 cycle 数

(3) 定量设计的原则:

① 大概元素事件优先的原则: make the common case fast

② Amdahl's Law.

$$\text{Speedup} = \frac{1}{\text{不可改} + \frac{\text{可改}}{\text{加速}}} \leq \frac{1}{\text{不可改}} = \frac{1}{1-f_e}$$

→ 只针对一个时, 加速不超过 $\frac{1}{1-f_e}$

(4) benchmark of SPEC 2017

采用几何平均

Chapter 2 计算机中的数

1. 整型数

1) 补码: ① 零唯一, 负数比正数多一个

② 是以 2^{n+1} 为模的计量系统

③ $[X+Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}}$ 符号位可以直接加

2) 无符号整数: $x = x_0 x_1 x_2 \dots x_n$ 范围是 $0 \leq x \leq 2^{n+1} - 1$ 模

与有符号数转化: 可以是星式 or 表达式中有 unsigned 或 unsigned

需要正整数的模或用数的各位编码表示集合, 用 unsigned

3) 按位运算与逻辑运算

→ & 1: 按位与/或

→ (可以看作对集合的操作 & 交 | 并 ^ 对称差 ~ 补)

&&, ||, !

4) 位扩展、截断、移位

① 扩展: 0 扩展: 向高位扩展时全填 0 (针对 unsigned)

符号位扩展: 把符号位扩展

② 截断: 强行将长数转换为短数时

→ 直接截断、重新解释结果

移位量大于 word size 时

$w=32$ $x \ll y$ ③ 移位: $\Leftrightarrow x \ll (y \bmod 32)$

左移 $\ll$ 不分算术及逻辑, 抛弃最高位填 0

右移 $\gg$ 逻辑右移: 右移一位, 左填 0
signed 算术右移: 右移一位, 若符号为 1 左补 1

(5) 整数的加减运算

eg. int x, z
short y

A. 无符号加法: 直接丢弃最高进位量 ($u+v = (u+v) \bmod 2^n$)
补码加法: 共用同上, 直接丢弃

x = 127

y = -9

z = x + y

B. 补码减法: $[X - Y]_{\text{补}} = [X]_{\text{补}} + [-Y]_{\text{补}}$

无符号数减法: 共用
 $u - v = u - v + 2^w = u + (2^w - v)$
 $= u + [v + 1]$

(6) 溢出的判断

正+正=正 / 正-负=负
负+负=负 / 负-正=正
产生了溢出

法1: 符号位判断 (直接就用上方)

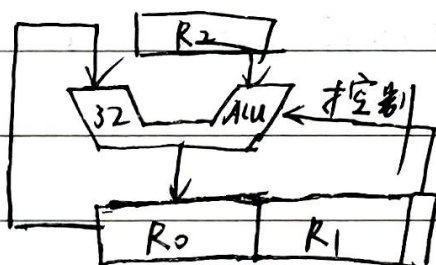
法2: 最高位进位与次高位进位不同则溢出

法3: 将正数符号位计为 00
负数符号位计为 11
运算结果非 00/11 则溢出

(7) 定点数乘法的实现原理

原码乘①可以直接用级联的乘法电路。

②可以用加法器实现



逻辑右移

补码乘法：① 转换原码 — 同 — 转换回来

② Booth 算法

y_{i+1}	y_i	
0	0	无
1	0	减 x
1	1	无
0	1	加 x

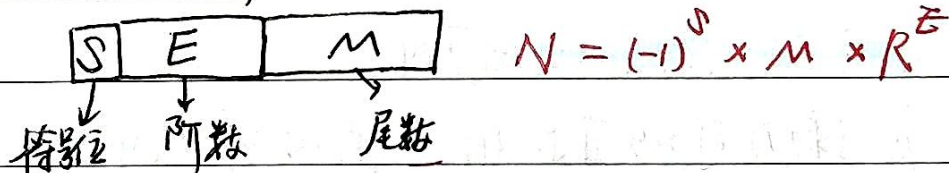
(8) 整数乘法

相同 { ① unsigned : $u \times v = (u \cdot v) \bmod 2^w$
 共用乘法单元 ② 补码乘法：
 直接编码相乘取低位 (是取模的意思)

$u \ll k \Leftrightarrow u \times 2^k$ 对无符号、带符号都有效
 乘法可能溢出

2. 浮点数

(1) 浮点数的编码

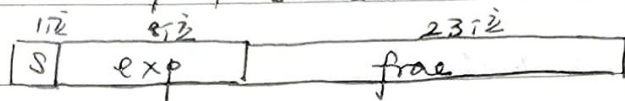


↑ 范围 ↑ 精度 ↑

规格化数：尾数最高为 1 的数 $13.25 = 0.1101001$

尾数在 $\frac{1}{2} \sim 1$ 之间

IEEE 754 浮点数



$$(-1)^s \times 1.m \times 2^{e-127}$$

① 尾数部分: ① 采用原码表示

② 规格化: ① XXX... (缺省右补0)

阶码: ① 非零, 非全1, 即 01H ~ FEH

② 采用移码 -126 ~ +127

③ 分类: float (32 bits) 1 + 8 + 23

double (64 bits) 1 + 11 + 52

高精度 (80 bits) 1 + 15 + 63 or 64

float 符号位 阶码 尾数 表示

0/1 255 1xx 非零 0xx NaN (发信号)

0/1 255 0 +∞/-∞

0/1 1~254 f $(-1)^s \times (1.f) \times 2^{e-127}$

非规格化数 $\Leftarrow$ 0/1 0 f (非0) $(-1)^s \times (0.f) \times 2^{(e-126)}$

0/1 0 0 +0/-0

③ 表示范围

规格化数

单精

$$1.0 \times 2^{1-127}$$

max

$$2^{127} \times (2 - 2^{-23})$$

双精

$$1.0 \times 2^{1-1023}$$

$$2^{1023} \times (2 - 2^{-52})$$

绝大多可以用 unsigned Integer 比较器直接比较

非规格化 vs. 规格化

规格化 vs. 0

除了: 要先比较符号位 (负为1)
-0 = 0 : NaN 最大?

B) 浮点数的算术运算

① 加法 (减)

对齐 $\rightarrow$ 尾数加减 $\rightarrow$ 规格化 $\rightarrow$ 舍入 $\rightarrow$ 检查溢出
 $\rightarrow$ 不精确, ex. 大 + 小, 小 + 小相加, 大 + 大

舍入方法 (round to even)

不是 0.5 时为四舍五入, 是 0.5 时记尾数为 even

如果对齐后相差 $\Delta E \geq 25$ 就不用^{继续}运算了

② 浮点数的类型转换

double/float $\rightarrow$ int (~~截去小数~~)

$\rightarrow$ 若为 Inf/NaN, 变为 int_{min} = -2147...

int $\rightarrow$ double 精确

int $\rightarrow$ float round to even 保留

Chapter 3. MIPS 指令系统

1. MIPS 指令简介

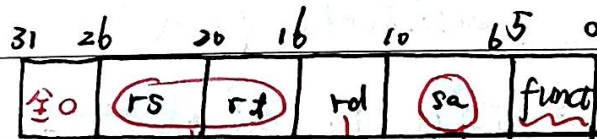
- ① 特征: ① 简单: 指令定长, 数量少, 格式简单 (I, J, R)
- ② 寻址方式少: 基址 + 16位偏移量 / 立即数寻址 / 寄存器寻址

② MIPS 通用寄存器

0	\$zero	常量 0	\$t8-\$t9	temp.
1	\$at	汇编器 temp.	\$k0-\$k1	操作, 系统内核
2-3	\$v0-\$v1	函数返回结果, 表达式值	\$gp	Global Pointer
4-7	\$a0-\$a3	函数调用临时参数	\$sp	Stack
8-15	\$t0-\$t7	Temp.	\$fp	Frame Pointer
	\$s0-\$s7	saved temp	\$ra	return address

③ MIPS 指令格式. (MIPS 指令系统操作码是不定长的)

① R-type:

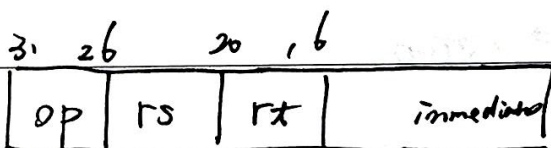


源寄存器 目标寄存器

左移位指令中代表偏移量 (其余不用)

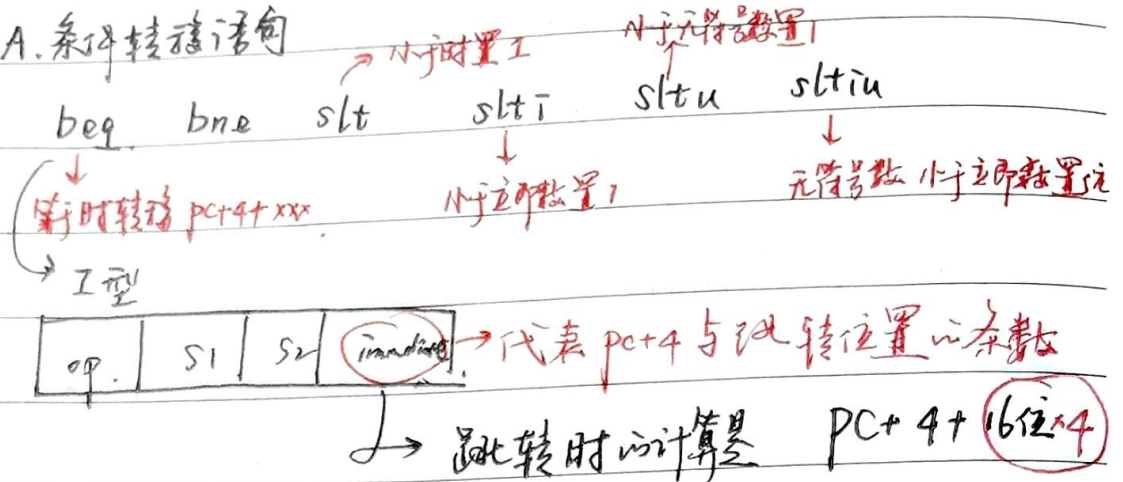
此时 25-21 为 rs 不用

② I-type



eg. lw \$t0, 24(\$s3)

A. 条件转移语句

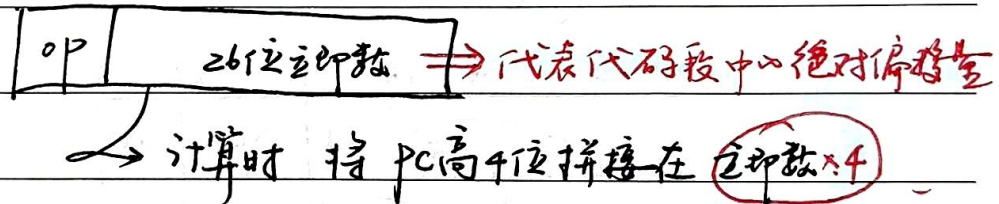


B. 无条件转移语句:

a. $Jal + 10000$ 将 $pc+4$ 放入 $\$ra$ } 类似函数调用

b. $jr \$ra$ $go to \$ra$

c. $J 10000$: 直接 $go to xxx$



2. MIPS 过程调用指令

1) 涉及的指令

jal ; $jr \$ra$

$\downarrow$ $\downarrow$

将 $pc+4$ 放入 $\$ra$ $\rightarrow$ return

12) 执行的六个步骤:

参数传递 ($\$a0 - \$a3$) $\rightarrow$ jal $\rightarrow$ 申请资源 $\rightarrow$ 执行
 $\rightarrow$ 返回值存储 ($\$v0 - \$v1$) $\rightarrow$ 返回控制权 (jr $\$ra$)

13) MIPS 寄存器分类:

① 过程调用时保存: $\$ra, \$sp, \$gp, \$fp, \$s0 - \$s7, \$a0 - \$a3$

② 不用保存: 如 $\$t0 - \$t7$

3. MIPS 伪指令

14) 为了方便编程, MIPS 还提供伪指令:

Chapter 4. MIPS 单周期处理器设计

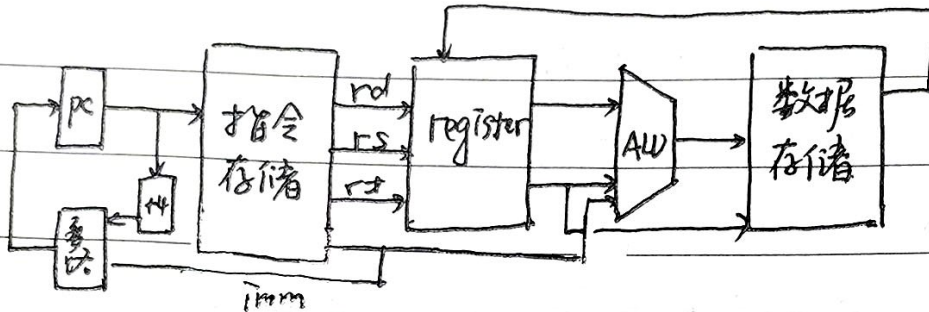
1. 概况:

(1) 设计目标: MIPS 指令的子集 add, sub, ori

lw, sw; beq, j

(2) 设计步骤 (见 PPT)

二. 数据通路



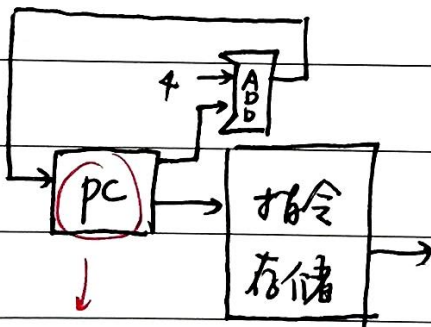
(1) 单周期处理器: 一个时钟周期完成一条指令

状态单元每周期只更新一次

分为产生数据值的组合单元及包含状态与时序状态单元
如 ADD, Decoder
如寄存器
ALU, MUX
存储器

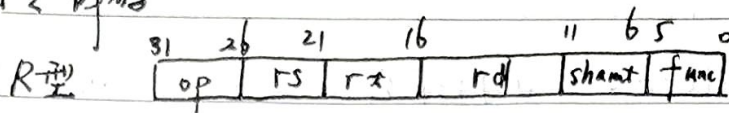
A) 处理器的数据通路.

A. 取指令

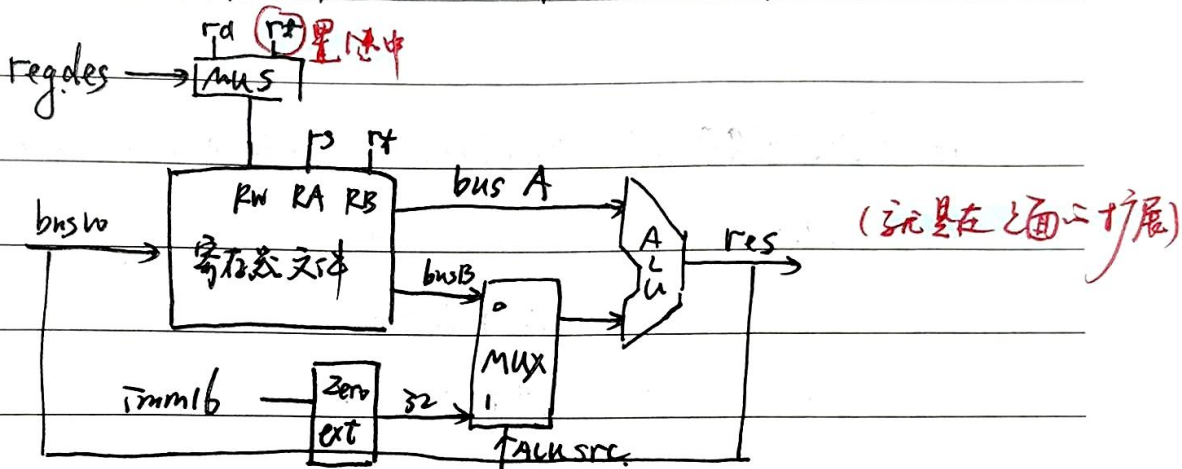
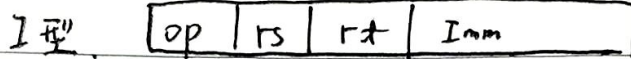
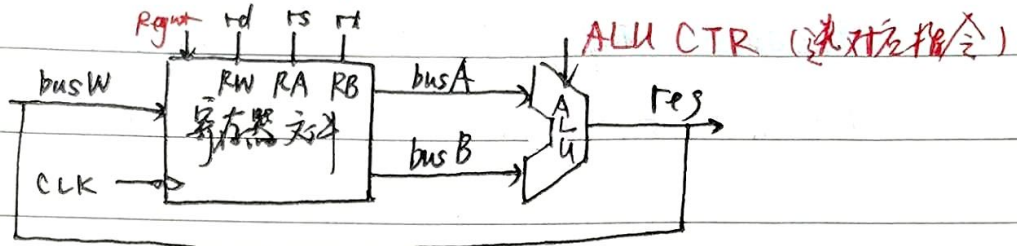


是时序电路, 要在新时钟开始后再次

B. 指令译码



先把 op & func 送入, 译出对应控制信号



lw & sw 见 PPT.

Jump. 直接左移 2 位, 与 PC+4 的高 4 位相连.

(3) 控制信号:

ALUctr: 运算操作码

Jump: 是否为无条件

ALUSrc: ALU数据选择

MemWrite: 存储器写

Ext op: 无/带符号扩展

Mem to Reg: ~~从~~ mem/ALU输出

Branch: 是否为条件

Regwriter Reg Dest: register

(4) 集成控制信号 (见PPT吧, 实在不知如何做笔记).

Chapter 5. 微程序设计

1. 机器指令的阶段.

取指令 → 译码 → 取 register → ALU 计算 → 访存写回 → 下一条指令地址

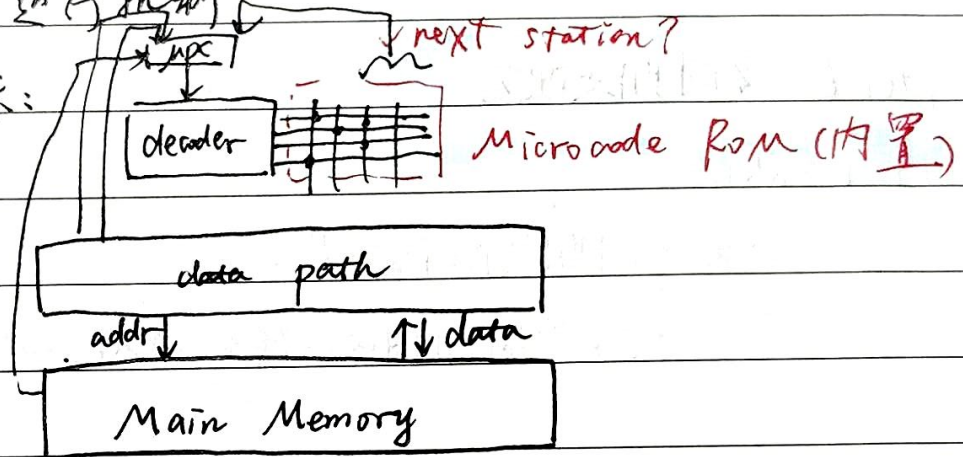
↓ CPU 可分为 { 控制器
数据通路.

由于 ROM 更快 & 比纯逻辑电路便宜

→ 在 CPU 内安一个 μ CPU

2. 微程序的运行机制

(1) 图示:



(2) 微指令的流程.

复杂指令也不需要更改

线路了, 只用编写对应该 program

A. 取指阶段 — 执行阶段

PC 给内存, PC+1 → 等待内存 → dispatch on opcode

先执行, 然后 goto instruct fetch

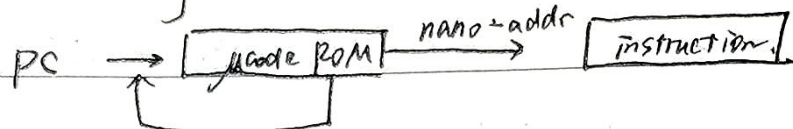
B. μ PC = { next μ PC+1
dispatch 看 opcode
fetch/spin → 等 mem
true/false → cond jmp (to fetch)

3. μ Code 的设计

① 水平型: 每条微指令有多个微操作并行
更快, 但 ROM 面积大 (位数宽)

② 垂直型: 每条微指令代表一个数据通路操作
宏指令需要更多微指令, 更慢

③ Nanocoding: (二级索引)



Chapter 6. 多周期处理器

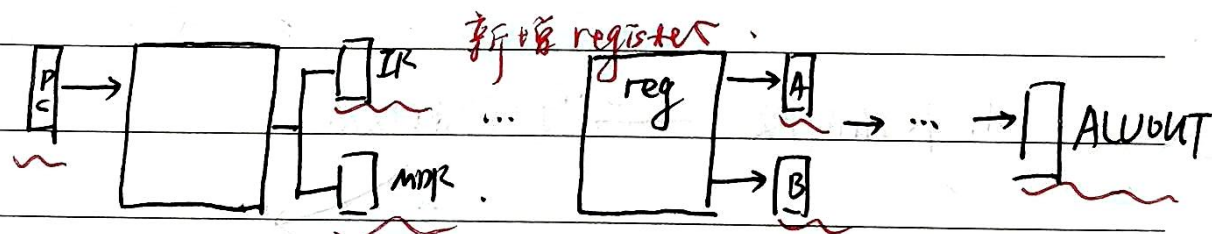
1. 单周期缺陷:

① 时钟周期受限于最慢的一条

② 一条指令周期内的功能部件只能执行一次

2. 多周期处理器

(1) 专用通路的单周期 CPU



多周期时钟周期更长 (比单)

完成一条指令时间变长

时钟周期 vs 机器周期 (CPU 周期) vs 指令周期

↓
从内存取出一条指令的最短时间

Chapter 6. 流水线处理器

1. MIPS 五阶段流水线

(1) Ifetch $\rightarrow$ Reg/Dec $\rightarrow$ Exec $\rightarrow$ Mem $\rightarrow$ Wr

- $\rightarrow$ 易流水化:
- ① 所有指令等长
 - ② 指令格式少 (都可以在第二阶段读 reg)
 - ③ 只有 lw/sw 要 mem (也和其他一样要 exe)
 - ④ 每条指令只用写一个结果

当前 CPI = 1

(2) 指令流水线的实现方式 (+增加寄存器)

① 取指阶段:

取完之后将 指令长 pc+4 放入 IF/ID 段中

② 译码段:

将指令输入控制器得出所有信号

然后逐级传递

可以马上算出来接下来地址

是否要记? $\times$ 不用 32	Ifetch 不用	
64	Dec 用 ExtOP / <u>JMP</u>	↑ 发生跳转 前面全清空
147	Exec 用 ALUSrc / ALUop / Regdst	
106	Mem 用 MemWr / Branch	
71	Wr 用 Mem Mem to Reg / RegWr	

可能的冒险

二、相关性和冒险

(1) 简化表示



(2) 相关性 & 冒险

① 结构冒险

资源相关性 (所需部件为之前工作)

解决方法: a. 增加部件 (如指令 cache & 数据 cache)

b. 插入 NOP

c. 功能单元流水化

在寄存器文件的读写时 (写口在 CLK 前半段, 读口在 CLK 后半段) ★★

② 数据冒险

数据相关性 (等待读写)

大致分两种, 一种 register 的, 一种读 mem-use 的

解决方法:

a. 编译器插入 NOP

b. 流水线停顿

停顿检测电路

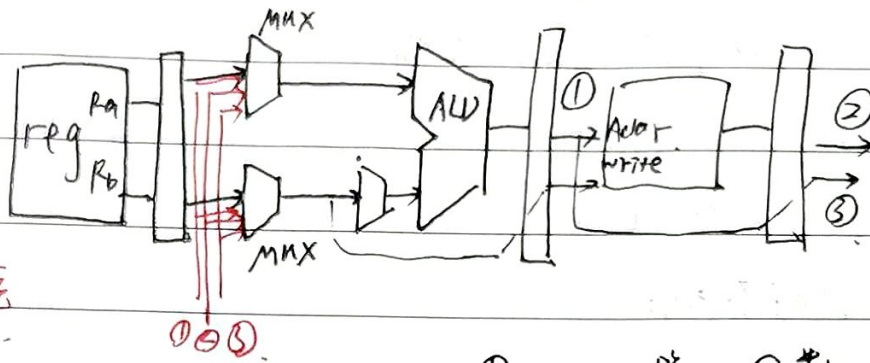
如果源 = 目 & write enable

(看 rs, rt, & rd) , 造成 stall, 卡住后面的指令

c. 前向传递 (forwarding)

→ 不等写回 reg. 就直接写入

只有lw之后
要stall一下给
序下来



重要记录

① ALU计算 ② 数据读出 ③ 上阶段

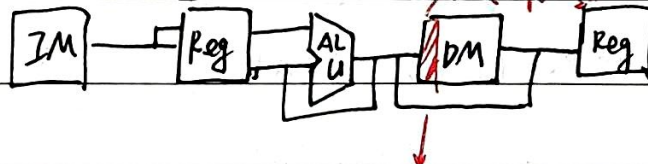
控制信号和 b. 差不多

③ 控制冒险

a. jump 指令

→ CPI=2. 直接把后一条 flush.

b. beq 指令



这才知道 → CPI=4

①

可以将 beq 向前移动 (增加 加法器)

一边译码. 一边 beq 算

过时 ← ② 转移延迟槽

→ 把一定会执行的放在槽中

现代心:

编译器决定.

① 多发射

② 长流水

③ 乱序 (如做div时 add可用)

④ 多线程

3. 转移预测

(1) 预测流水的问题:

① 开销及复杂度增加

② 性能下降 (指令在缓存 $\uparrow$ 下) 相关性 $\uparrow$ stall 更多

③ 功耗大

(2) 转移预测:

方向: ① 静态 (不管后面是啥都预测某一方向)

② 动态:

→ $\begin{cases} \text{一位} \\ \text{二位饱和} \end{cases}$

ST	WT	WNT	SNT
11	- 10	- 01	- 00

③ G-share.

局部相关 vs 全局相关

地址: BTB (target buffer)

→ 存 pc (当前及跳转), 分支历史

pc $\rightarrow$ BTB $\begin{cases} \text{存在:} \begin{cases} \text{分支成功} \\ \text{否: 删除 BTB 对应项} \end{cases} \\ \text{不存在:} \begin{cases} \text{不分支} \\ \text{分支: 存进 BTB} \end{cases} \end{cases}$

Chapter 7. Cache

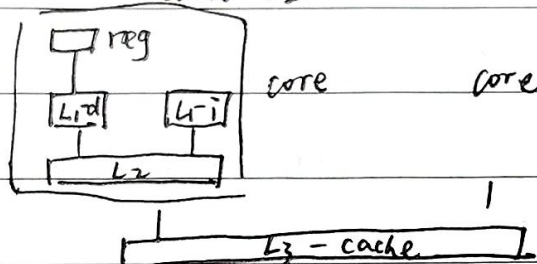
1. Cache 的基本原理

(1) 局部性原理

temporal locality: 最近访问的项目, 不久后也会访问

spatial locality: 与当前访问项目地址邻近项目可能被访问

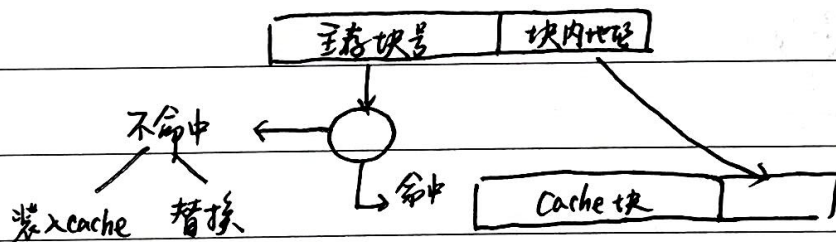
(2) cache 的层次性



cache 是以内存块

block 的方式组织的

(3) cache 的结构原理 (硬件管控, 虚拟地址)



2. 主存与 cache 的地址映射

(1) 直接映射

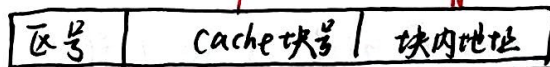
(每组只有 1 way)

(主存的块有且仅有一个 cache 的对应位置)

快, 简单
但 hit 率低

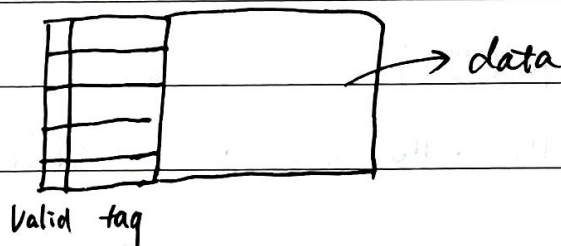
$$i = j \bmod n$$

主存地址



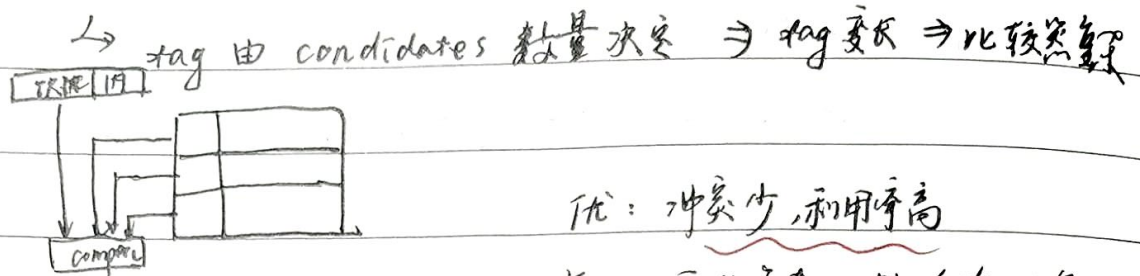
cache地址

cache



(只有一组)

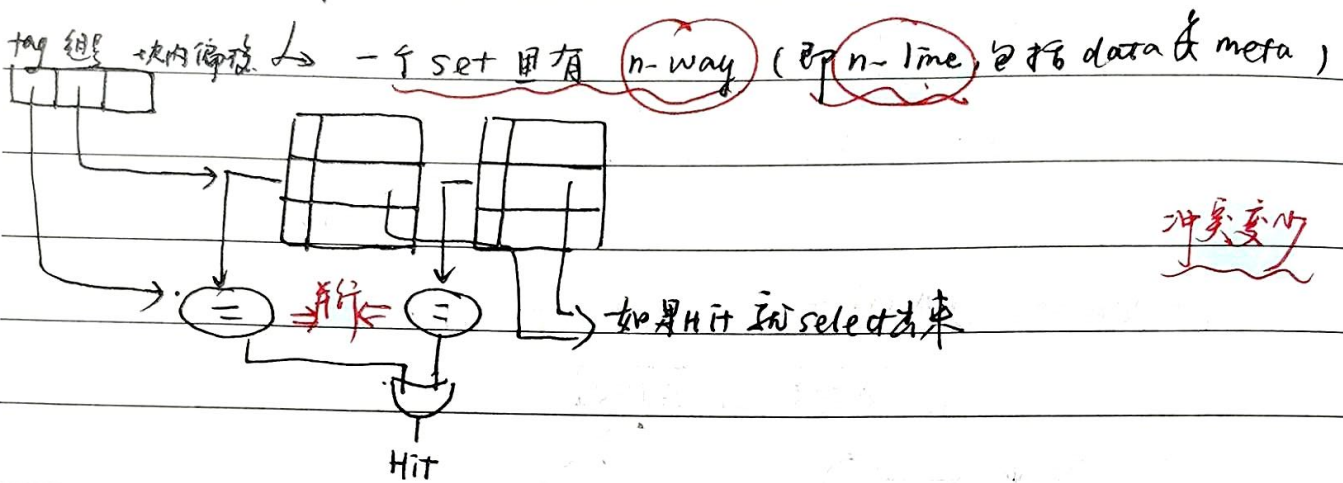
(K) 全相联映射 (主存每一块能在任一 cache 块中被)



优: 冲突少, 利用率高

缺: 硬件复杂, 替换算法复杂

(J) 组相联映射



3. cache 更新策略

(N) 写策略 (保证 cache 与 memory 一致)

① write-through (写穿) $\Rightarrow$ 更安全

不一致时 既写回 cache 也直接写回 memory

$\rightarrow$ 可以用 write buffer, 不仅是调节 cpu 时间

也可将多次相邻地址放一起, 批量写

② write-back (写回) $\Rightarrow$ 频繁写时更快

只写回 cache, 不写 mem, 只有替换时写回

$\rightarrow$ 设置脏位, 修改过才写 mem.

cache 失效时的写策略

① 按写分配法 (与 write back 搭配) → 保证最新在 cache
一般是按块 ← miss → 调入 cache → 修改 → 写回 cache

② 不按写分配法 (与 write-through 搭配)

miss → 绕过 cache 直接写 memory

4. 替换算法

① LRU :

↳ 硬件复杂 (如 4-way 就要 $4 \times \log_2 4$ 位)
颠簸现象

Chapter 8. VLIW 与超标量.

1. 超长指令字流水线

(1) 由软件打包多条互相独立的指令打包发射.

(2) 如 ALU/branch 与 数据传递的双发射
的 VLIW.

✓
循环展开

寄存器编号消除相关性

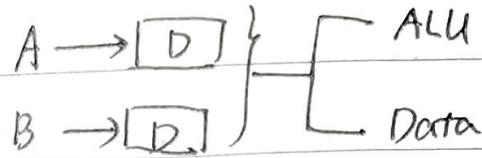
↳ 硬件简单, 成本低能耗少

编译复杂, 代码膨胀

锁步机制

2. 按序超标量流水线

(1) 示意



用一个硬件判断其两条指令是否可以同时

(2) 可以自动判断是走哪条线

是否有结构冒险或数据冒险

静态调度, 编译器也能尽量避免上述冒险

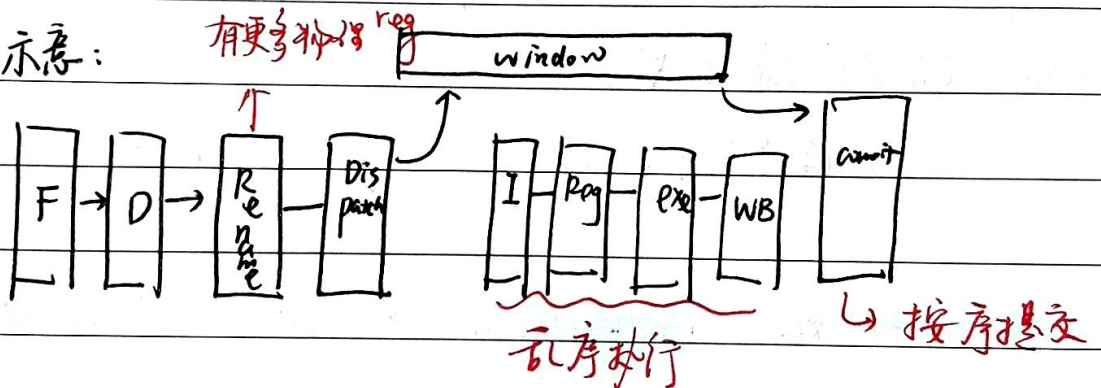
3. 乱序超标量流水线

(1) 问题:

名字引起的假相关, 如 (WAW & WAR)

改名即可

(2) 示意:



可以看 slides 上的 Buffer 示例

用于改名及按序提交

Lecture 9. 开发各级并行性

1. 指令级并行

(1) 原因: 指令间没有依赖, 天然有并行性

→ 流水线技术, 多发射

衡量指标 CPE (cycle per element)

(2) 优化:

功能单元有 latency

⇒ 循环展开

减少转移预测

→ 利用空间局部性

→ 增加指令性并行

Throughput Bound

受限于 func. & load..

2. 数据级并行 (SIMD 子字并行)

→ 一条 instruction 处理多个 data

(1) CPU 内部新增向量寄存器

搭配 SIMD 的指令

→ 减少条数, 增加单条指令译码的效率.

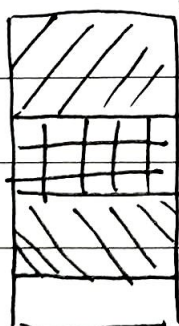
(2) AVX (一种指令集)

3. 线程级并行

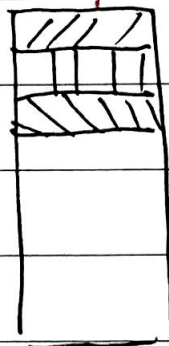
(1) 同时多线程

都是有套 register

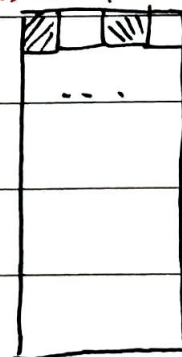
保证每个
线程+线程



粗粒度



细粒度



同时多线程

→ 有改名

⑦ 假共享:

① 对称 & 不对称存储

② cache-一致性 — 写作废 / 写更新

③ cache block 的假共享, (可能改同一-block不同位置)

→ 多线程不应定义在相邻结构